

Lecture 7: The Krusell–Smith Method

第七讲：Krusell–Smith 方法

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 19, 2026

University of Toronto

Recap & Framing

Where we are 我们走到哪里了

- L1–L4: build the household block; solve Aiyagari at a single steady state.
- L5: HouseholdStages.jl + the deterministic *transition* between two steady states.
- L6: spatial heterogeneity + indirect inference (calibration).
- **Today (L7):** aggregate *uncertainty*.
- L1–L4: 搭家庭块; 求单点 Aiyagari 稳态。
- L5: HouseholdStages.jl 与两稳态之间的确定性过渡。
- L6: 空间异质性 + 间接推断 (校准)。
- **今天 (L7):** 总量不确定性。

What changes today

今天有什么不同

- In L5, the path $\{K_t\}$ was *deterministic*: once we conjectured it, the household solve never had to integrate over the future of K .
- Today we add an aggregate productivity shock Z_t on a Markov chain. Prices (r_t, w_t) become *stochastic* along the path.
- Two new ideas: a **state explosion** and the classical **K-S workaround**.
- L5 中 $\{K_t\}$ 是确定的：一旦猜出路径，家庭无需对未来 K 求期望。
- 今天加入总量生产率冲击 Z_t (Markov 链)。价格 (r_t, w_t) 沿路径变随机。
- 两个新想法：**状态空间爆炸**与经典的 **K-S 折中**。

Today's roadmap

今日路线图

1. Recap & framing.
2. The state explosion.
3. The K-S workaround.
4. *Break*.
5. Implementing K-S in code.
6. The accuracy debate (R^2 , Den Haan).
7. Wrap; L8 teaser.

1. 回顾与定位。
2. 状态空间爆炸。
3. K-S 折中。
4. 中场休息。
5. K-S 的代码实现。
6. 准确性辩论 (R^2 、Den Haan)。
7. 收尾；预告 L8。

One sentence preview

一句话预告

The rational household needs the entire wealth distribution Λ_t in its state space.

That's infeasible. **Krusell–Smith** replace Λ_t with a single number (K_t) plus a one-line forecast rule. Then they show that — against all expectations — this is essentially exact in the baseline calibration.

The *why* of that exactness is the topic of §6.

理性主体的状态空间应包含整个财富分布 Λ_{t0}

这不可行。**Krusell–Smith** 把 Λ_t 换成单个数 K_t 加一行预测规则；随后他们证明——出乎意料——在基线标定下这几近精确。这种“精确”的原因是 §6 的主题。

State Explosion

Setup 设定

Aggregate state. TFP $Z_t \in \{Z_{\text{bad}}, Z_{\text{good}}\}$ on a 2-state Markov chain P_Z .

Idiosyncratic state. Income $y_t \in \{y_{\text{lo}}, y_{\text{hi}}\}$ on Markov chain P_y , plus wealth b_t .

Technology. Cobb–Douglas with TFP Z_t :

$$Y_t = Z_t K_t^\alpha L^{1-\alpha}.$$

L is the stationary-mean of y (constant in this model since $P_y \perp Z$).

总量状态。 TFP $Z_t \in \{Z_{\text{bad}}, Z_{\text{good}}\}$ ，二态 Markov 链 P_Z 。

个体状态。 收入 $y_t \in \{y_{\text{lo}}, y_{\text{hi}}\}$ (Markov P_y)，加财富 b_{t0} 。

生产。 Cobb–Douglas + TFP Z_t :

$$Y_t = Z_t K_t^\alpha L^{1-\alpha}.$$

L 是 y 的平稳均值 (P_y 与 Z 独立，所以是常数)。

Prices on the realised path 实现路径上的价格

By firm optimisation, the rental rate and wage are

$$r(Z_t, K_t) = \alpha Z_t (K_t/L)^{\alpha-1} - \delta,$$

$$w(Z_t, K_t) = (1 - \alpha) Z_t (K_t/L)^\alpha.$$

So once we know the realised (Z_t, K_t) , the household sees deterministic prices *this* period.

The trouble is that the household must also forecast *next* period's (Z_{t+1}, K_{t+1}) when choosing b_{t+1} .

按厂商优化，租金率与工资为

$$r(Z_t, K_t) = \alpha Z_t (K_t/L)^{\alpha-1} - \delta,$$

$$w(Z_t, K_t) = (1 - \alpha) Z_t (K_t/L)^\alpha.$$

只要 (Z_t, K_t) 已实现，本期价格就是确定的。

困难在于：选 b_{t+1} 时，家庭还要预测下期 (Z_{t+1}, K_{t+1}) 。

Why K_{t+1} depends on Λ_t

为什么 K_{t+1} 依赖 Λ_t

K_{t+1} is the integral of household savings choices:

$$K_{t+1} = \int b'(b, y; \cdot) d\Lambda_t(b, y).$$

The policy $b'(\cdot)$ depends on the prices the household *expects* for next period, which depend on K_{t+1} , which depends on the integral above, which depends on Λ_t .

Conclusion. Two distributions Λ_t with the same mean K_t can in principle generate different K_{t+1} .

K_{t+1} 是家庭储蓄选择的积分：

$$K_{t+1} = \int b'(b, y; \cdot) d\Lambda_t(b, y).$$

储蓄策略 $b'(\cdot)$ 依赖于家庭对下期价格的预期，预期依赖于 K_{t+1} ， K_{t+1} 又依赖于上面这个积分——依赖于 Λ_t 。

结论。 两个均值 K_t 相同的分布 Λ_t 原则上可以给出不同的 K_{t+1} 。

The rational Bellman

理性 Bellman

A fully rational household tracks the whole state $(b, y; \Lambda, Z)$:

$$V(b, y; \Lambda, Z) = \max_{c, b'} u(c) + \beta \mathbb{E}[V(b', y'; \Lambda', Z') \mid y, Z],$$

subject to

$$c + b' = (1 + r(Z, K(\Lambda))) b + w(Z, K(\Lambda)) y,$$

and the law of motion

$$\Lambda' = T(\Lambda, Z, Z').$$

完全理性家庭追踪全状态 $(b, y; \Lambda, Z)$:

$$V(b, y; \Lambda, Z) = \max_{c, b'} u(c) + \beta \mathbb{E}[V(b', y'; \Lambda', Z') \mid y, Z],$$

预算约束

$$c + b' = (1 + r(Z, K(\Lambda))) b + w(Z, K(\Lambda)) y,$$

分布演化

$$\Lambda' = T(\Lambda, Z, Z').$$

Λ is infinite-dimensional

Λ 是无穷维的

Λ_t is a probability measure on (b, y) . To pin it down we need to know its value on every measurable set — an infinite-dimensional object.

To use $V(b, y; \Lambda, Z)$ in code, we would have to tabulate V over a grid in Λ . But Λ does not live in a finite-dimensional grid.

This is the Krusell–Smith problem.

Without a workaround, full-rationality HA + aggregate shocks is computationally hopeless.

Λ_t 是 (b, y) 上的概率测度。要确定它，需要在每个可测集上的值——无穷维对象。

要在代码里用 $V(b, y; \Lambda, Z)$ ，得在 Λ 的网格上制表 V 。但 Λ 不住在任何有限维网格里。

这就是 Krusell–Smith 问题。

不做近似，”完全理性 HA + 总量冲击”在计算上无望。

Just how bad is it? 究竟有多糟糕?

Discretise Λ naively. Suppose we cut wealth into $N_w = 80$ bins and income into $N_y = 2$ levels. A distribution on this grid is a point in the $(160-1)$ -simplex — 159 free coordinates.

To grid V over Λ with even 5 points per coordinate: 5^{159} cells. About 10^{111} .

For comparison. Atoms in the observable universe: $\sim 10^{80}$.

A coarser grid still loses to the universe.

对 Λ 朴素离散化。 财富分 $N_w = 80$ 格、收入 $N_y = 2$ 状态，则分布是 $(160-1)$ -单纯形上的一点——159 个自由坐标。

每个坐标 5 点制表 V : 5^{159} 个格子，约 10^{111} 。

对照。 可观测宇宙中的原子数约 10^{80} 个。更粗的网格依然输给宇宙。

Two ways out

两条出路

(a) **Reduce the state.** Believe Λ_t is summarised by a few moments. Replace Λ with those moments in the state. Solve the reduced problem.

(b) **Reduce the representation.** Keep Λ in the state, but parameterise $V(\Lambda, \cdot)$ by some flexible family of functions whose parameter dimension does not depend on Λ 's grid.

Today. Path (a) — Krusell–Smith.

Tomorrow (L8). Path (b) — neural-net parameterisation.

(a) **减状态。**假设 Λ_t 可由少数矩概括，把矩放入状态，解约化问题。

(b) **减表示。**保持 Λ 在状态中，但用一族灵活函数参数化 $V(\Lambda, \cdot)$ ，其参数维度与 Λ 网格无关。

今天。(a) — Krusell–Smith。 **明天 (L8)。**(b) — 神经网络参数化。

The Krusell–Smith problem statement

K–S 问题的正式陈述

Given.

- Aggregate shock Z_t on P_Z .
- Idiosyncratic shock y_t on P_y , independent of Z .
- Technology $Y_t = Z_t K_t^\alpha L^{1-\alpha}$, depreciation δ .
- Preferences: CRRA, discount β , no borrowing.

Find. A recursive equilibrium: V , $b'(\cdot)$, prices (r, w) , and a law of motion for Λ .

Concession. Today's equilibrium is *approximate* — rationality bounded to a

给定。

- 总量冲击 Z_t (P_Z)。
- 个体冲击 y_t (P_y)，与 Z 独立。
- 生产 $Y_t = Z_t K_t^\alpha L^{1-\alpha}$ ，折旧 δ 。
- 偏好：CRRA、贴现 β 、不能借贷。

求。递归均衡： V 、 $b'(\cdot)$ 、价格 (r, w) 、 Λ 的演化律。

让步。今天的均衡是近似的 —— 理性被限制在 Λ 的低维概括上。

K–S Workaround

Bounded rationality

有限理性

Krusell and Smith (1998): *drop* the assumption that the household tracks Λ_t .

Tell the household:

"You are allowed to know K_t (a number) and Z_t . You are not allowed to know any other moment of Λ_t ."

This is a *modelling* choice — it makes households behave *as if* they cannot observe the full distribution.

It will turn out that, in this baseline, full-rationality decisions are essentially identical to bounded-rationality decisions.

But that's the result, not the assumption.

Krusell–Smith (1998): 放弃家庭追踪 Λ_t 的设定。告诉家庭：

" 你只允许知道 K_t (一个数) 和 Z_t 。 Λ_t 的其他矩你都不许看。 "

这是一个建模选择——让家庭表现得仿佛无法观察整体分布。

后面会发现：在基线下，完全理性与有限理性决策几乎相同。但这是结果，不是假设。

The Approximate Law of Motion

近似演化律

The household is given a forecast rule for K :

$$\log K_{t+1} = a_0(Z_t) + a_1(Z_t) \log K_t.$$

Two coefficients per Z state. For 2-state Z , four numbers total.

Log-linear. An empirical insight from KS-1998: when one tries this on simulated data, log-linear in K is already excellent. We will see the *why* in §6 (approximate aggregation). For now, take it as given.

家庭被赋予 K 的预测规则：

$$\log K_{t+1} = a_0(Z_t) + a_1(Z_t) \log K_t.$$

每个 Z 状态两个系数。 两态 Z 共四个数。
对数线性。 KS-1998 的经验观察：在模拟数据上拟合时，对数 K 的线性已极佳。为什么见 §6（近似聚合）。现在先承认。

The bounded-rationality Bellman

有限理性 Bellman

Replace Λ with K in the state:

$$V(b, y, Z, K) = \max_{c, b'} u(c) + \beta \mathbb{E}[V(b', y', Z', K')],$$

subject to

$$c + b' = (1 + r(Z, K)) b + w(Z, K) y,$$

with the agent's *belief*

$$K' = \exp(a_0(Z) + a_1(Z) \log K),$$

and Z' drawn from P_Z .

The state is now (b, y, Z, K) : **4D**, finite, gridable

状态中用 K 代替 Λ :

$$V(b, y, Z, K) = \max_{c, b'} u(c) + \beta \mathbb{E}[V(b', y', Z', K')],$$

预算

$$c + b' = (1 + r(Z, K)) b + w(Z, K) y,$$

家庭信念

$$K' = \exp(a_0(Z) + a_1(Z) \log K),$$

Z' 从 P_Z 抽取。

状态变为 (b, y, Z, K) : **4 维**、有限、可制表。 15

The chain in five stages

五个 Stage 的链

Within a period, in time order:

1. IncomeShock: $y_t \rightarrow y_{t+1}$ (Markov on :income).
2. IncomeReceipt: $b' \leftarrow (1+r)b + wy$, with (r, w) from (Z, K) .
3. ConsumptionSavings: pick b_{t+1} .
4. KEvolve: $K_t \rightarrow K_{t+1}$ via the agent's ALM (deterministic).
5. ZShock: $Z_t \rightarrow Z_{t+1}$ (Markov on :Z_idx).

期内 (时间顺序):

1. IncomeShock: $y_t \rightarrow y_{t+1}$ 。
2. IncomeReceipt: $b' \leftarrow (1+r)b + wy$, 价格读自 (Z, K) 。
3. ConsumptionSavings: 选 b_{t+1} 。
4. KEvolve: $K_t \rightarrow K_{t+1}$, 按家庭 ALM (确定)。
5. ZShock: $Z_t \rightarrow Z_{t+1}$ 。

IncomeShock ○ IncomeReceipt ○ ConsumptionSavings ○ KEvolve ○ ZShock.

Why KEvolve and ZShock are at the end 为什么 KEvolve 与 ZShock 在末尾

IncomeReceipt reads prices from the cell's (Z, K) coordinates.

If KEvolve / ZShock ran *before* receipt, the cell would already carry next period's (Z, K) when receipt fires, and prices would be misaligned.

Putting them at the end keeps receipt's reading of (Z, K) on the *start-of-period* values — as the economics calls for.

A small detail, but recurring when wiring stages into chains.

IncomeReceipt 通过 cell 坐标 (Z, K) 读价格。

若 KEvolve / ZShock 在它之前跑，receipt 触发时 cell 上的 (Z, K) 已是下期值，价格会错位。

放末尾可保 receipt 读到的是期初 (Z, K) ——符合经济意义。

小细节，但在链组装中经常遇到。

The K-S iteration in words

K-S 迭代 (文字版)

1. **Pick** an initial ALM $(a_0^{(0)}, a_1^{(0)})$.
2. **Solve** the 4D household VFI given the current ALM.
3. **Simulate** a long economy under realised $\{Z_t\}$ and *true* aggregation $K_t = \int b d\Lambda_t$.
4. **Refit** (a_0, a_1) by OLS of $\log K_{t+1}$ on $\log K_t$, separately per Z .
5. **Iterate** until ALM coefficients stop moving.

1. **初始化** ALM $(a_0^{(0)}, a_1^{(0)})$ 。
2. **解当前** ALM 下的 4D VFI。
3. **模拟**：用实现的 $\{Z_t\}$ 与真实聚合 $K_t = \int b d\Lambda_t$ 。
4. **回归**：分 Z 状态对 $\log K_{t+1}$ 关于 $\log K_t$ 做 OLS。
5. **迭代至** ALM 系数收敛。

Why iterate

为什么要迭代

The first VFI uses some *guessed* ALM. The household's saving choices follow that belief.

When we simulate the economy with these choices and aggregate honestly, the resulting K path may not be log-linear with the guessed coefficients.

Refit. Update beliefs to the empirical regression. Re-solve.

Fixed point. Stop when the ALM the household believes is the ALM the household generates. That's the K-S equilibrium.

首轮 VFI 用猜测的 ALM。家庭储蓄按该信念。

按该选择模拟并真实聚合，所得 K 路径未必恰好满足猜测的对数线性关系。

回归更新：把信念换成经验回归。再解。

不动点：家庭相信的 ALM = 家庭生成的 ALM。这就是 K-S 均衡。

Inner solve, outer solve

内层与外层

Inner. A standard 4D VFI at a fixed ALM. The household block from `HouseholdStages.jl` does the heavy lifting; nothing here is K-S-specific.

Outer. The new piece: simulate, regress, repeat.

Compare:

- L4 outer loop: scalar $\bar{K}$ via tatonnement.
- L5 outer loop: path $\{K_t\}$ via tatonnement.
- L7 outer loop: four ALM coefficients

内层。固定 ALM 下的标准 4D VFI；包负责繁重工作，无 K-S 特定逻辑。

外层。新东西：模拟、回归、迭代。

对照：

- L4 外层：对 $\bar{K}$ 做 tatonnement。
- L5 外层：对路径 $\{K_t\}$ 做 tatonnement。
- L7 外层：用 OLS 更新四个 ALM 系数。

Break

Break

中场休息

10 min

After the break: the K-S algorithm in code.

休息后：K-S 算法的代码实现。

Implementing K–S

Parameters

参数

Canonical K-S calibration (1998):

- $\beta = 0.96$, $\gamma = 1$ (log utility),
 $\alpha = 0.36$, $\delta = 0.025$.
- $Z \in \{0.99, 1.01\}$ with P_Z persistent
(0.875 on-diagonal).
- $y \in \{0.07, 1.0\}$.

Grids (small, for pedagogical speed):

- $N_w = 80$ on log-spaced $[0, 80]$.
- $N_K = 5$ on linear $[10.5, 14.5]$.
- Cell count: $80 \times 2 \times 2 \times 5 = 1600$.

经典 K-S 标定 (1998):

- $\beta = 0.96$ 、 $\gamma = 1$ (对数效用)、
 $\alpha = 0.36$ 、 $\delta = 0.025$ 。
- $Z \in \{0.99, 1.01\}$ ， P_Z 高持续 (对角
0.875)。
- $y \in \{0.07, 1.0\}$ 。

网格 (教学速度，刻意取小):

- $N_w = 80$ ，对数网格 $[0, 80]$ 。
- $N_K = 5$ ，线性网格 $[10.5, 14.5]$ 。
- Cell 数： $80 \times 2 \times 2 \times 5 = 1600$ 。

State layout

状态空间

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(p.w_min, p.w_max;  
                                       length = p.N_w, spacing = :log)),  
    StateAxis(:income, p.y_grid),  
    StateAxis(:Z_idx, [1, 2]),  
    StateAxis(:K,      continuous_grid(p.K_min, p.K_max;  
                                       length = p.N_K, spacing = :linear)),  
)
```

- Two continuous axes (:wealth, :K), two categorical (:income, :Z_idx).
- Same layout the simulator will use, modulo the chain we run on it.
- 两连续维 (:wealth、:K)，两离散维。
- 模拟时复用此 layout，仅运行的链不同。

Stages 1–2: IncomeShock and IncomeReceipt

Stage 1–2

```
income_shock = MarkovStage(layout; axis = :income, transition = p.P_y)

income_receipt = WealthChangeStage(layout;
  wealth_axis = :wealth,
  wealth_post = (cell; env) -> begin
    Z = p.Z_vals[cell.Z_idx]
    pr = ks_prices(cell.K, Z, p)
    (1 + pr.r) * cell.wealth + pr.w * cell.income
  end,
)
```

Note. The receipt closure reads (Z, K) from the cell coordinates, not from env. So prices vary across cells *within* a single solve.

注意。 receipt 闭包从 cell 坐标读 (Z, K) ，而非 env；价格在同一次求解内随 cell 变化。

Stage 3: ConsumptionSavings

Stage 3

```
savings = ConsumptionSavingsStage(layout;  
   $\beta$  = p. $\beta$ ,  
  utility = (cell, c; env) -> u_crra(c, Val(p. $\gamma$ )),  
  wealth_axis = :wealth,  
  monotone_search = :divide_conquer,  
)
```

- Same stage as Aiyagari / spatial.
- Utility is per-cell — but here only depends on c .
- Divide-and-conquer search exploits monotonicity in wealth.
- 与 Aiyagari / 空间一致。
- 此处效用仅依赖 c 。
- 分治搜索利用财富方向的单调性。

Stages 4–5: KEvolve and ZShock

Stage 4–5

```
K_evolve = WealthChangeStage(layout;  
  wealth_axis = :K,  
  wealth_post = (cell; env) -> begin  
    a0 = env.a0[cell.Z_idx]  
    a1 = env.a1[cell.Z_idx]  
    exp(a0 + a1 * log(cell.K))  
  end,  
)  
  
Z_shock = MarkovStage(layout; axis = :Z_idx, transition = p.P_Z)
```

KEvolve is a *deterministic* mass-redistribution stage on :K, reading the ALM coefficients from env.

KEvolve 是 :K 上的确定性质量再分配 stage, 从 env 读 ALM 系数。

Compose; declare moments

组链；声明矩

```
chain = income_shock ◦ income_receipt ◦ savings ◦ K_evolve ◦ Z_shock

hh = define_moments!(chain;
  K_supplied = at_end(integrand = :wealth, reduce = sum))
```

- `K_supplied` sums b over the end-of-period Λ — the integral that appears in the K-S iteration.
- Used for the deterministic Aiyagari SS at $Z = 1$ *and* for diagnostics during the K-S loop.
- `K_supplied` 对期末 Λ 沿 b 求和——K-S 迭代中要用的积分。
- 用于 $Z = 1$ 的确定性 Aiyagari 稳态以及 K-S 循环的诊断。

Inner solve: VFI at a candidate ALM

内层：给定 ALM 解 VFI

```
a0_init = [log(K_bar), log(K_bar)]  
a1_init = [0.0,          0.0]  
env0     = make_env(hh; a0 = a0_init, a1 = a1_init)  
  
ss = solve_steady_state_given_env!(hh, env0;  
                                   lambda_tol = 1e-5,  
                                   lambda_maxiter = 50_000)
```

First guess. A *constant* ALM: $K_{t+1} = \bar{K}$ for both Z states ($a_1 \equiv 0$, $a_0 \equiv \log \bar{K}$).

Cost. 400 inner VFI iterations on the 1600-cell grid.

首猜。 常数 ALM：两 Z 状态均设 $K_{t+1} = \bar{K}$ 。

开销。 约 400 次内层 VFI (1600 cells)。

Simulation: why a separate chain

模拟：为什么要单独的链

When we simulate we want *realised* (Z_t, K_t) , not the agent's belief.

The chain's KEvolve and ZShock encode the *belief*: they would push K along the ALM and Z along P_Z , regardless of what actually realised.

Solution. Build a second chain — just the first three stages (IncomeShock ◦ IncomeReceipt ◦ ConsumptionSavings) — and drive its (Z, K) from outside.

模拟时希望用实现的 (Z_t, K_t) ，而非家庭信念。

链中的 KEvolve 与 ZShock 编码信念：会按 ALM 推 K 、按 P_Z 推 Z ，不顾实际实现。

解决。另建一链——仅前三 stage (IncomeShock ◦ IncomeReceipt ◦ ConsumptionSavings) ——从外部驱动它的 (Z, K) 。

One simulation step

单步模拟

Maintain Λ_t as a 2D distribution on (b, y) .
Each period:

1. Compute $K_t = \sum_{b,y} b \Lambda_t[b, y]$.
2. Lift Λ_t to a 4D distribution concentrated at (Z_t, K_t) — share-redistributed between the two adjacent K-grid points.
3. Push the 4D distribution through the sim chain (3 stages).
4. Marginalise out (Z, K) to get Λ_{t+1} on (b, y) .
5. Draw $Z_{t+1} \mid Z_t$ from P_Z .

把 Λ_t 维持为 (b, y) 上的 2D 分布。每期：

1. 算 $K_t = \sum_{b,y} b \Lambda_t[b, y]$ 。
2. 提升 Λ_t 到 4D，在 (Z_t, K_t) 处集中 — 邻近 K-grid 两点按权重分配质量。
3. 推 4D 分布过模拟链（3 stage）。
4. 在 (Z, K) 上边际化得 Λ_{t+1} 。
5. 由 P_Z 抽 $Z_{t+1} \mid Z_t$ 。

Seating the simulation chain

模拟链的初始化

```
hh_sim = ks_household_sim(p)
# Run backward on the sim chain with the converged V_ss as
# terminal value, so its savings policy is materialised.
backward!(hh_sim, copy(ss.V), (;))
```

- The sim chain has no KEvolve/ZShock, so it does not need ALM coefficients in env.
- backward! fills the savings policy from the inner-solve V^* .
- Then ks_simulate can push Λ forward.
- 模拟链无 KEvolve/ZShock, env 无需 ALM 系数。
- backward! 用 V^* 填充储蓄策略。
- 然后 ks_simulate 即可推 Λ 向前。

Refit the ALM by OLS

用 OLS 重拟 ALM

```
function regress_ALM(K_path, Z_idx_path, z_idx, window)
    rows = [t for t in window if Z_idx_path[t] == z_idx]
    x = log.(K_path[rows])
    y = log.(K_path[rows] .+ 1)
    # OLS:  $y = a_0 + a_1 * x$  ;  $R^2 = 1 - SS_{res}/SS_{tot}$ 
    ...
    return (; a0, a1, R^2, n)
end
```

```
fit_bad = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
fit_good = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)
```

Two regressions (one per Z). Burn-in 500, fit-window the rest.

两次回归 (每个 Z 一次)。预热 500 期，余下用于拟合。

The outer loop

外层循环

```
function ks_iterate(p, K_bar,  $\Lambda_0$ ; damp = 0.5, tol = 1e-3, maxiter = 15)
    hh, hh_sim = ks_household(p), ks_household_sim(p)
    a0, a1 = [log(K_bar), log(K_bar)], [0.0, 0.0]
    for it in 1:maxiter
        _ss = solve_steady_state_given_env!(hh, make_env(hh; a0, a1); ...)
        backward!(hh_sim, copy(_ss.V), (;))
        sim = ks_simulate(hh_sim, p, T_sim;  $\Lambda_{\text{init}} = \Lambda_0$ , Z_init_idx = 1)
        fit_b = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
        fit_g = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)
        a0_n, a1_n = [fit_b.a0, fit_g.a0], [fit_b.a1, fit_g.a1]
         $\Delta$  = maximum(abs.(vcat(a0_n .- a0, a1_n .- a1)))
         $\Delta$  < tol && return (; a0, a1, sim, converged = true, iters = it)
        a0 = (1 - damp) .* a0 .+ damp .* a0_n
        a1 = (1 - damp) .* a1 .+ damp .* a1_n
    end
end
```

Running it 运行

Convergence behaviour.

- Converges in **13 outer iterations** to $\Delta < 10^{-3}$.
- Inner VFI: ~ 400 iters each outer round, falling as we approach the fixed point.
- Mean K over the post-burn-in window: ≈ 13.35 .

Damping 0.5 on the coefficient update prevents the first few rounds from overshooting.

收敛表现。

- **13 轮外迭代**收敛到 $\Delta < 10^{-3}$ 。
- 内层 VFI 约 400 轮每外轮，接近不动点时下降。
- 预热后窗口的 K 均值 ≈ 13.35 。

外层系数更新阻尼 0.5，避免最初几轮过冲。

Accuracy

The converged ALM

收敛 ALM

After 13 outer iterations:

$$\log K_{t+1} = 0.31 + 0.88 \log K_t \quad (Z = \text{bad}),$$

$$\log K_{t+1} = 0.35 + 0.86 \log K_t \quad (Z = \text{good}).$$

Coefficients on $\log K_t$ are close (0.88 vs 0.86); intercepts differ (0.31 vs 0.35) and pick up the level shift between recession and boom.

Just four numbers describe the dynamics of K across all states of the wealth distribution.

13 轮外迭代后:

$$\log K_{t+1} = 0.31 + 0.88 \log K_t \quad (Z = \text{bad}),$$

$$\log K_{t+1} = 0.35 + 0.86 \log K_t \quad (Z = \text{good}).$$

$\log K_t$ 系数接近 (0.88 vs 0.86); 截距差 (0.31 vs 0.35) 捕捉繁荣-衰退间的水平差。四个数字描述 K 在所有财富分布状态下的动态。

The shocking result: $R^2 \approx 1$

关键结果: $R^2 \approx 1$

The converged ALM regression:

- $Z = \text{bad}$: $R^2 = 0.99850$.
- $Z = \text{good}$: $R^2 = 0.99737$.

Reading this. A linear function of $\log K_t$ explains the variation in $\log K_{t+1}$ within each Z state *essentially completely*.

Implication. The full distribution Λ_t adds no usable information beyond its mean K_t . The household that knows only K_t forecasts as well as the household that knew all of Λ_t .

收敛 ALM 回归:

- $Z = \text{bad}$: $R^2 = 0.99850$.
- $Z = \text{good}$: $R^2 = 0.99737$.

解读。 $\log K_t$ 的线性函数几乎完全解释了每个 Z 状态下 $\log K_{t+1}$ 的变化。

含义。 Λ_t 在其均值 K_t 之外不提供可用信息。只知 K_t 的家庭与知 Λ_t 的家庭预测一样准。

Approximate aggregation

近似聚合

Property. K_t 's dynamics are described to high accuracy by a function of (K_t, Z_t) alone.

Why here?

- Cobb–Douglas + patient agents + CRRA utility.
- Savings policies $b'(b, y; Z, K)$ are *approximately linear* in b in the relevant range.
- Then $K_{t+1} = \int b' d\Lambda_t$ is determined by K_t ; higher moments of Λ_t near-irrelevant.

性质。 K_t 动态可用 (K_t, Z_t) 的函数高精度刻画。

为什么这里成立？

- Cobb–Douglas + 耐心主体 + CRRA 效用。
- 储蓄策略 $b'(b, y; Z, K)$ 在相关区间近似线性于 b 。
- $K_{t+1} = \int b' d\Lambda_t$ 由 K_t 决定； Λ_t 高阶矩几乎无关。

Krusell & Smith (1998), §3.2。

The Den Haan critique

Den Haan 批评

R^2 averages residuals over many periods — it can hide *systematic drift*: a small one-step bias compounds over the long run.

Den Haan (2010). A sharper diagnostic.

Starting from a common K_0 :

- Compute the true K_t path by aggregating household decisions every period.
- Compute an ALM-only path:
 $K_{t+1} = \exp(a_0(Z_t) + a_1(Z_t) \log K_t)$, no aggregation.

Compare over hundreds of periods.

R^2 在多期上取均值 — 可能掩盖系统漂移：每期的小偏差长期累积。

Den Haan (2010)。更严格的诊断。给定 K_0 ：

- 算“真实” K_t 路径：每期真实聚合家庭决策。
- 算“仅 ALM”路径：
 $K_{t+1} = \exp(a_0(Z_t) + a_1(Z_t) \log K_t)$ ，不聚合。

比较数百期。

What we see 我们看到了什么

In this calibration, over 800 post-burn-in periods:

$$\max_t \left| \frac{K_t^{\text{ALM}} - K_t^{\text{true}}}{K_t^{\text{true}}} \right| \approx 0.28\%.$$

The ALM-only path tracks the true path to within a fraction of a percent throughout the simulation.

Interpretation. The bounded-rationality assumption is essentially exact here. K-S agents forecast as well as fully rational ones — not just on average, but trajectory-by-trajectory.

本标定下，预热后 800 期内：

$$\max_t \left| \frac{K_t^{\text{ALM}} - K_t^{\text{true}}}{K_t^{\text{true}}} \right| \approx 0.28\%.$$

仅-ALM 路径在整段模拟中偏离真实路径不到 0.3%。

解读。 此处有限理性假设几乎完美 — K-S 家庭与全理性家庭的预测不止平均一致，逐路径也一致。

When K-S works, when it breaks

K-S 何时成立、何时失效

K-S works when:

- Savings near-linear in wealth.
- Agents relatively patient.
- No large MPC heterogeneity.

K-S can break when:

- Occupation choice / extensive-margin entry.
- Fat-tailed income shocks.
- Sharp non-CRRA preferences (habits, loss aversion).
- Default with curvature;
inequality-shape dynamics

K-S 成立:

- 储蓄策略近似线性于财富。
- 主体相对耐心。
- 无大幅 MPC 异质。

K-S 失效:

- 职业选择 / 外延边际进入。
- 收入冲击肥尾。
- 非 CRRA 偏好（习惯、损失厌恶）。
- 含曲率违约；不平等-形状动态。

What the linear ALM cannot see

线性 ALM 看不到的东西

The ALM compresses Λ_t into a scalar. Structural changes that move wealth between agents with different MPCs — or shift mass between regimes (constrained vs unconstrained) — without moving K_t much are *invisible* to the ALM.

In the baseline calibration these movements happen to have small impact on K_{t+1} . In richer models they do not.

This is the wedge L8 will exploit.

ALM 把 Λ_t 压成一个标量。在不同 MPC 主体间挪财富、或在约束 vs 未约束等状态间挪质量，却不大幅改变 K_t 的结构变化，对 ALM 不可见。

基线下这些移动对 K_{t+1} 影响小；更丰富模型则不然。

这正是 L8 要利用的缝隙。

Wrap

What we built today

今天搭出了什么

- Identified the **state explosion** when aggregate shocks enter HA.
 - Adopted the K-S **bounded-rationality** ansatz: agents see K_t , not Λ_t .
 - Postulated a **log-linear ALM** for K_{t+1} given (K_t, Z_t) .
 - Iterated VFI $\rightarrow$ simulate $\rightarrow$ regress to a fixed point.
 - Found $R^2 > 0.997$ on the converged ALM, and a Den Haan error $< 0.3\%$.
- 识别了加入总量冲击带来的**状态爆炸**。
 - 采用 K-S 的**有限理性**假设：家庭只看 K_t 、不看 Λ_t 。
 - 设定**对数线性** ALM 预测 K_{t+1} 。
 - 迭代 VFI $\rightarrow$ 模拟 $\rightarrow$ 回归至不动点。
 - 收敛 ALM 的 $R^2 > 0.997$ ，Den Haan 误差 $< 0.3\%$ 。

Tomorrow: drop the ALM

明日：放弃 ALM

The K-S baseline works because the wealth distribution happens to be unimportant.

What if it weren't?

L8: Continuation Value is All You Need.

- Replace V on a grid with a neural net $V_\theta(b, y, Z, K)$.
- Drop the parametric ALM and the regression step.
- Train V_θ on the Bellman residual — the "Pooled Bellman" loss.

Same stage chain, different inner solver.

K-S 基线之所以成立，是因为财富分布“恰好”无关紧要。**若不然呢？**

L8: Continuation Value is All You Need.

- 把 V 的网格表换成神经网络 $V_\theta(b, y, Z, K)$ 。
- 放弃参数化 ALM 与回归步。
- 在 Bellman 残差上训练 V_θ — “Pooled Bellman” 损失。

链结构一致，内层求解器变。

Open Q&A

自由问答

?